Pretende-se que desenvolva um serviço (cliente e servidor) de *chat* (conversação) muito simplificado sobre UDP.

É esperado que os clientes possam enviar frases (obtidas do *standard input* através da função `fgets()`) com, no máximo, 49 caracteres. Uma frase é um conjunto de caracteres terminado por `'\n'`. A frase `"quit\n"` deve ser interpretada pela aplicação cliente como um pedido do utilizador para terminar.

Como resposta a uma frase recebida, o servidor deve enviar sempre, e apenas ao seu remetente, uma única mensagem com as últimas frases recebidas (4 no máximo), incluindo a frase que esse cliente lhe acabou de enviar. No entanto cada uma dessas frases deve ser enriquecida, no seu início, com o endereço IP e porto (i.e. "coordenadas") do cliente que a enviou. Ou seja, o servidor deve de ir guardando localmente as frases recebidas num formato aumentado, para que comecem sempre pelas coordenadas do seu remetente e só depois tenham o conteúdo efetivamente recebido. O servidor não precisa que as mensagens que envia lhe sejam confirmadas.

Os utilizadores colocam habitualmente o seu nome em todas as frases que enviam para simplificar a sua identificação. Tal não é relevante para a sua resolução, mas ajuda a compreender a seguinte ilustração de fluxo (repare que apenas depois de Ana enviar uma segunda frase é que recebe a primeira frase do José):

Emissor	Frase	Recetor
Cliente 1	"Ana: Ola.\n"	Servidor
Servidor	"127.0.0.1:62199# Ana: Ola.\n"	Cliente 1
Cliente 2	"Jose: Ola.\n"	Servidor
Servidor	"127.0.0.1:62199# Ana: Ola.\n 127.0.0.1:62204# Jose: Ola.\n"	Cliente 2
Cliente 1	"Ana: Alguem ligado?\n"	Servidor
Servidor	"127.0.0.1:62199# Ana: Ola.\n 127.0.0.1:62204# Jose: Ola.\n 127.0.0.1:62199# Ana: Alguem ligado?\n"	Cliente 1
Cliente 1	"Ana: Ola Jose, tudo bem?\n"	Servidor
Servidor	"127.0.0.1:62199# Ana: Ola.\n 127.0.0.1:62204# Jose: Ola.\n 127.0.0.1:62199# Ana: Alguem ligado?\n 127.0.0.1:62199# Ana: Ola Jose, tudo bem?\n"	Cliente 1
Cliente 2	"Jose: Ola Ana, esta ok!\n"	Servidor
Servidor	"127.0.0.1:62204# Jose: Ola.\n 127.0.0.1:62199# Ana: Alguem ligado?\n 127.0.0.1:62199# Ana: Ola Jose, tudo bem?\n 127.0.0.1:62204# Jose: Ola Ana, esta ok!\n"	Cliente 2
Cliente 1	"quit\n"	
Cliente 2	"Jose: E contigo?\n"	Servidor
Servidor	"127.0.0.1:62199# Ana: Alguem ligado?\n 127.0.0.1:62199# Ana: Ola Jose, tudo bem?\n 127.0.0.1:62204# Jose: Ola Ana, esta ok!\n 127.0.0.1:62204# Jose: E contigo?\n"	Cliente 2
Cliente 2	"quit\n"	

1) [40%] Desenvolva o cliente.
   Veja a estrutura sugerida abaixo e as respetivas alíneas desta pergunta – todas possuem a mesma cotação.
2) [50%] Desenvolva o servidor.
   Veja a estrutura sugerida e as respetivas alíneas desta pergunta – todas possuem a mesma cotação.
3) [10%] Explique o que aconteceria se existissem mais do que 4 clientes ligados.

Nota 1:

`char *fgets(char *str, int numChars, FILE *stream)`

To read from the standard input pass `stdin` as the `stream`.

The `fgets` function reads a string from the input stream argument and stores it in `str`.

`fgets` reads characters from the current stream position to and including the first newline character, to the end of the stream, or until the number of characters read is equal to `numChars-1`, whichever comes first.

The result stored in is appended with a `NULL` character. The newline character, if read, is included in the string.

Nota 2:

`int strcpy_s(char *dest, const char *src)`

The `stpcpy()_s` function (same as `strcpy`) copy the string `src` to `dst` (including the terminating `'\0'` character).

Nota 3:

`int strcmp (const char *s1, const char *s2)`

The `strcmp()` function lexicographically compare the null-terminated strings `s1` and `s2`.

Return an integer greater than, equal to, or less than 0, according as the string `s1` is greater than, equal to, or less than the string `s2`.

Nota 4:

`int strcat_s(char *s1, char *s2)`

The `strcat_s()` function (same as `strcat`) append a copy of the null-terminated string `s2` to the end of the null-terminated string `s1`, then add a terminating `'\0'`. The string `s1` must have sufficient space to hold the result.

```
// ==================================== UDP Chat Client ====================================
// Exemplo de linha de comando usado para contactar um servidor local UDP no porto 6000:
// chat_client.exe 127.0.0.1 6000
// ==========================================================================================

#include <stdio.h>
#include <winsock2.h>
#include <ws2tcpip.h>

void Abort(const char*msg);

// ---------------------------------------- main ----------------------------------------

int main(int argc, char* argv[]) {
   SOCKET sockfd;
   int res, phrase_len, nbytes;
   struct sockaddr_in serv_addr;
   WSADATA wsaData;

   // a) Declare a variável phrase destinada a guardar a frase introduzida pelo utilizador.
   // Não esqueça que a mesma deve prever espaço para guardar o zero terminador de qualquer
   // string. Sugere-se que use a mesma variável para guardar a resposta do servidor. Preveja
   // uma dimensão adequada. Defina/inicialize outras variáveis que entenda necessárias.
   /*..A..*/ char phrase[256];

   // ========== WINSOCKS LIBRARY INITIALIZATION ==========
   res = WSAStartup(MAKEWORD(2, 2), &wsaData);
   if (res != 0) {
       fprintf(stderr, "WSAStartup failed: %d\n", res);
       exit(EXIT_FAILURE);
   }

   // ========== CREATE A SOCKET TO SEND/RECEIVE DATAGRAMS ==========
   // b) Crie um socket UDP.
   sockfd = /*..B..*/;  socket(AF_INET, SOCK_DGRAM,0)
   if (sockfd == INVALID_SOCKET)
       Abort("Unable to obtain a socket from operating system!");

   // ========== SETUP THE SERVER ADDRESS ==========
   memset((char*)&serv_addr, 0, sizeof(serv_addr)); // Coloca a zero todos os bytes
   serv_addr.sin_family = AF_INET; //Address Family: Internet
   res = inet_pton(AF_INET, argv[1], &serv_addr.sin_addr);  // IP do servidor
   if (res != 1)
       Abort("The IP value is invalid.");

   // c) Preencha o campo relativo ao porto de destino pretendido, passado através da linha
   // de commando.
   serv_addr.sin_port = /*..C..*/;     htons(atoi(argv[2]);
```

```
   // ========== SEND THE MESSAGE TO THE SERVER ==========
   While (1) {
       fprintf(stdout, "Introduza mensagem: ");
       fflush(stdout);
       // d) Utilize a função fgets() para ler a frase introduzida no stdin e guardar na
       // variável definida phrase. Lembre-se que a frase a guardar deve estar limitada a um
       // máximo de 49 carateres, '\n' incluído. Se a frase colhida pelo fgets() não contemplar
       // o '\n' (porque o utilizador introduziu mais que 48 caracteres) deve ter o cuidado
       // de incluí-lo na última posição de phrase.
       /*..D..*/

       // e) Obtenha o número de bytes (comprimento) da frase
       phrase_len = /*..E..*/;

       // f) O cliente deve terminar quando o utilizador introduzir a frase "quit\n".
       /*..F..*/

       // g) Envie a frase introduzida para o servidor.
       res = /*..G..*/;
       if(res == SOCKET_ERROR)
           Abort("The operating system is not able to send the datagram.");

       // ========== WAIT FOR THE ANSWER FROM THE SERVER ==========
       // h) Aguarde por resposta do servidor e coloque-a também na variável phrase
       nbytes = /*..H..*/;
       if (nbytes == SOCKET_ERROR || nbytes >= sizeof(phrase))
           Abort("Error while receiving the server answer.");
       phrase[nbytes] = '\0';
       fprintf(stdout, nbytes > sizeof(phrase)"\n%s\n", phrase);
   }

   // ========== CLOSE THE SOCKET ==========
   closesocket(sockfd);
   exit(EXIT_SUCCESS);
}

// ---------------------------------------- Abort ----------------------------------------
// Mostra uma mensagem de erro e o código associado ao último erro com Winsocks.
// Termina a aplicação com "exit status" a 1 (constante EXIT_FAILURE).
void Abort(const char* msg){
   fprintf(stderr, "<CLI> Fatal error: <%s> (%d)\n", msg, WSAGetLastError());
   exit(EXIT_FAILURE);
}
```

```
if (fgets(phrase, 49, stdin) != NULL) {
    size_t len = strlen(phrase);
    if (len > 0 && phrase[len - 1]!= '\n') {
        phrase[47] = '\n';
        phrase[48] = '\0';
    }
}
```

```
strlen(phrase)
```

```
if (strcmp(phrase, "quit\n") == 0) break;
```

```
sendto(sockfd, phrase, phrase_len, 0, (struct sockaddr*)&serv_addr, sizeof(serv_addr))
```

```
recvfrom(sockfd, phrase, sizeof(phrase) - 1, 0, (struct sockaddr*)&serv_addr, sizeof(serv_addr))
```

// ========== UDP Chat Server ==========

```
#include <stdio.h>
#include <winsock2.h>
#include <ws2tcpip.h>

#define SERV_UDP_PORT 6000
#define BUFFERSIZE 100

void Abort(const char* msg);

// ---------------------------------------- main ----------------------------------------
int main(int argc, char* argv[]) {
   SOCKET sockfd;
   int res, nbytes, length_addr, i;
   struct sockaddr_in serv_addr, cli_addr;
   char buffer[BUFFERSIZE];
   WSADATA wsaData;

   // i) Declare um array que permita guardar as últimas 10 frases recebidas. Lembre-se que
   // cada frase tem, no máximo, 49 caracteres. No entanto o servidor não guarda a frase tal
   // qual é recebida (ex. "Ana: Ola.") mas sim uma versão aumentada da mesma, que revela a
   // localização do seu autor (ex. "127.0.0.1:62199# Ana: Ola."). Este array deve funcionar
   // como uma estrutura FIFO (First In First Out). Ou seja, quando for recebida a 11ª frase
   // do diálogo, a 1ª recebida deve ser descartada por não caber; quando for recebida a 12ª
   // deve descartar-se a 2ª; e assim sucessivamente. A mensagem que o servidor remete ao
   // (último) cliente que o contata é composta pelas frases aumentadas alojadas nesse array,
   // concatenadas na sequência em que foram recebidas. Inicialize o array de forma adequada.
   // Defina/inicialize outras variáveis que entenda necessárias.
   /*..I..*/
```

```
char phrases[10][100] = {0};
int current_count = 0;
```

```
   // j) Declare uma string para guardar a concatenação das frases do array anterior. Esta
   // string, devidamente atualizada, é a mensagem que o servidor envia a cada cliente que o
   // contacte.
   /*..J..*/          char response_buffer[1000] = "";

   // ========== WINSOCKS LIBRARY INITIALIZATION ==========
   res = WSAStartup(MAKEWORD(2, 2), &wsaData);
   if (res != 0) {
      fprintf(stderr, "WSAStartup failed: %d\n", res);
      exit(EXIT_FAILURE);
   }

   // ========== CREATE A SOCKET TO SEND/RECEIVE DATAGRAMS ==========
   // k) Crie um socket UDP
   sockfd = /*..K..*/;          socket(AF_INET, SOCK_DGRAM, 0 )
   if (sockfd == INVALID_SOCKET)
       Abort("Unable to obtain a socket from operating system!");

   // ========== BINDS THE SOCKET TO THE SERVER ADDRESS ==========
   // Anuncie ao sistema que pretende receber datagramas vindos de qualquer interface de rede,
   // no porto pretendido
   memset((char*)&serv_addr, 0, sizeof(serv_addr));
   serv_addr.sin_family = AF_INET; // Address Family: Internet
   serv_addr.sin_addr.s_addr = htonl(INADDR_ANY); // Host to network long

   // l) Preencha o campo relativo ao porto local pretendido
   serv_addr.sin_port = /*..L..*/; // SERV_UDP_PORT  htons(SERV_UDP_PORT)

   // m) Associe o socket ao porto pretendido
   res = /*..M..*/;          bind(sockfd, (struct sockaddr*)&serv_addr, sizeof(serv_addr))
   if(res == SOCKET_ERROR)
       Abort("Operating System is currently unable to bind the socket.");

   // ========== ANSWER CLIENTS ITERACTIVELY ==========
   while (1) {
       fprintf(stderr, "<CHAT SERVER>Waiting new phrases from clients.\n");

       // n) Aguarde pela receção de um datagrama UDP no socket sockfd
       length_addr = sizeof(cli_addr);
       nbytes = /*..N..*/;recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0, (struct sockaddr*)&cli_addr, &length_addr)
       if (nbytes == SOCKET_ERROR)
           Abort("Error while waiting to receive a new datagram.");

       buffer[nbytes] = '\0';

       // o) Utilize a função sprintf_s() (o mesmo que sprintf()) para criar uma frase aumentada
       // constituída na sua primeira parte pelo endereço do cliente que a enviou seguida da
       // frase recebida, tal como definido no texto inicial. Pode utilizar a variável buffer
       // para armazenar temporariamente esta frase aumentada.
       /*..O..*/
        sprintf_s(buffer, sizeof(buffer), "%s:%d# %s", inet_ntoa(cli_addr.sin_addr), ntohs(cli_addr.sin_port), buffer)
       // p) Adicione a frase aumentada ao array definido em a). Este array deve funcionar
       // como um FIFO. Se guardar sempre a frase mais recente na primeira posição do array
       // ([0]), deve, antes de a adicionar, deslocar as frases anteriores uma posição acima.
       // Ou seja, a frase que ocupa a posição [3] transita para a posição [2], a que está em
       // [1] passa para [2], etc.). Repare que, caso o array esteja efetivamente cheio, este
       // processo conduzirá ao descarte (desejável) efetivo da frase mais antiga de entre as
       // guardadas e que ocupará sempre a última posição. O código seguinte pressupõe que
       // inicializou com uma string vazia ("") para todas as posições do array.
       for (i=3; i>0; i--)
           strcpy_s(/*..P..*/);   strcpy_s(phrases[i], sizeof(phrases[i]), phrases[i-1])

       // q) Guarde na string declarada em b) a concatenação de todas as frases do array a).
       /*..Q..*/  ****

       // r) Envie a mensagem (a concatenação mencionada atrás) ao cliente que o contatou.
       /*..R..*/
   }
}
       sendto(sockfd, response_buffer, strlen(response_buffer), 0, (struct sockaddr*)&cli_addr, length_addr)

// ---------------------------------------- Abort ----------------------------------------
// Código igual ao da função com o mesmo nome usado no cliente.
   ***

       response_buffer[0] = '\0';
       for (int j = 0; j < 10; j++) {
           if (phrases[j][0] != '\0') {
               strcat_s(response_buffer, sizeof(response_buffer), phrases[j]);
           }
       }
```